

TOOLS FOR DATA SCIENCE

Module 2 — Notes Summary

Compiled from my hand-written class notes

TABLE OF CONTENTS

1. Data Science Tools — Categories & Landscape	2
2. Open Source Tools for Data Science	3
3. Code & Data Asset Management, Dev & Execution Environments	4
4. Commercial & Cloud-Based Tools for Data Science	6
5. Languages of Data Science — Python, R & SQL	7
6. Inside a Relational Database	8
7. Libraries, APIs, Datasets & ML Models	9
8. Jupyter Notebook & JupyterLab	11
9. Git & GitHub — Version Control Basics	13
10. Git Core Workflow & Working with Branches	14

1. Data Science Tools — Categories & Landscape

1.1 Core Data Science Task Categories

Category	What It Does
Data Management	Collecting, storing & retrieving data efficiently and securely.
Data Integration & Transformation (ETL)	Combining data from multiple sources and converting it into a usable format.
Data Visualization	Representing data graphically (charts, plots, maps) for better understanding & decision-making.
Model Building	Training ML models to find patterns and make predictions.
Model Deployment	Integrating trained models into production systems for real-world use.
Model Monitoring & Assessment	Continuously evaluating model performance, accuracy & fairness after deployment.

1.2 Supporting Tasks

- **Data Asset Management:** organizing, storing & controlling access to data assets, with versioning & backup.
- **Code Asset Management:** managing code with version control, collaboration & change tracking (e.g. Git & GitHub).
- **Execution Environment:** platforms that run code with the required resources & libraries.
- **Development Environment (IDE):** tools used to write, test, debug & develop code.

Remember: every DS project = Core Tasks (Manage → Integrate → Visualize → Build → Deploy → Monitor) running on top of Supporting Tasks (Data/Code Asset Mgmt + Dev/Execution Environments).

2. Open Source Tools for Data Science

2.1 Data Management Tools

Type	Tools
Relational databases	MySQL, PostgreSQL
NoSQL databases	MongoDB, Apache CouchDB
File-based tools	Hadoop File System (HDFS)
Cloud file / search systems	Ceph, Elasticsearch — store & index data for fast search & retrieval

2.2 Data Integration & Transformation (ETL/ELT) Tools

Tool	Function
Apache Airflow	Manages & schedules ETL/ELT pipelines
Kubeflow	Runs ML pipelines on Kubernetes
Apache Kafka	Handles real-time data streaming
Apache NiFi	Visual tool for automating data flows
Apache Spark SQL	Processes large-scale structured data using SQL
Node-RED	Lightweight visual tool for data pipelines

2.3 Data Visualization Tools

Tool	Function
PixieDust	Python-based visualization with built-in UI support
Hue	Creates visualizations from SQL queries
Kibana	Visualization tool for Elasticsearch data
Apache Superset	Web-based dashboarding & exploration tool

2.4 Model Deployment & Monitoring Tools

Tool	Function
Apache PredictionIO	Deploys ML models (mainly Spark ML)
Seldon	Deploys models from multiple frameworks on Kubernetes
MLeap	Deploys Spark ML pipelines
TensorFlow Serving	Serves TensorFlow models in production
ModelDB	Tracks model experiments & metadata

Prometheus	General-purpose monitoring for systems / models
IBM AI Fairness 360	Detects & mitigates bias in models
Adversarial Robustness Toolbox	Tests model robustness against adversarial attacks

3. Code & Data Asset Management, Dev & Execution Environments

3.1 Code Asset Management

- **Git**: tracks & manages code changes (version control).
- **GitHub / GitLab / Bitbucket**: platforms for collaboration & version-control hosting.

3.2 Data Asset Management

- **Apache Atlas**: manages metadata & data lineage.
- **ODPi Egeria**: enables metadata sharing across platforms.
- **Kylo**: supports data ingestion & governance.

3.3 Model Development

- **IBM Watson Studio**: end-to-end platform for model development.

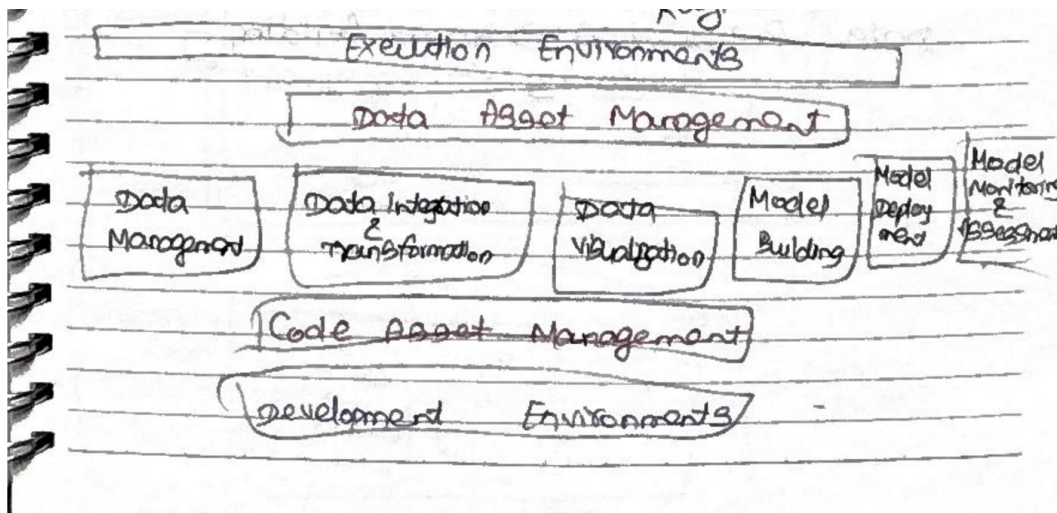
3.4 Development Environments (IDEs)

IDE	Visual / No-Code Tools
Jupyter Notebook, JupyterLab, Apache Zeppelin, RStudio, Spyder	KNIME, Orange

3.5 Execution Environments

- Apache Spark, Apache Flink, Ray — distributed computing engines that execute large-scale data science workloads.

3.6 How It All Fits Together



From my hand-written notes: Execution Environment and Development Environment run beneath Data/Code Asset Management, which in turn support the core task categories.

4. Commercial & Cloud-Based Tools for Data Science

4.1 Commercial Tools

Category	Tools
Database Management Systems	Oracle Database, Microsoft SQL Server, IBM Db2
Data Integration & Transformation	IBM InfoSphere data stage, Informatica, Talend
Data Visualization	Tableau, Microsoft Power BI, IBM Cognos Analytics
Model Building	SPSS, SAS Enterprise Miner
Development Environment	IBM Watson Studio Desktop

4.2 Cloud-Based Tools

- **Cloud platforms** = integrated Data Science platforms that cover the entire lifecycle.

Lifecycle covered: Data Preparation → Model Building → Deployment → Monitoring (all in one platform).

- **Examples:** IBM Watson Studio, Microsoft Azure ML.

5. Languages of Data Science — Python, R & SQL

5.1 Python vs R

Python	R
Widely used, general-purpose language	Used mainly for statistics
“Open-source” — focus on free technical development perspective.	“Free software” philosophy — focus on user freedom (Free Software Foundation concept)

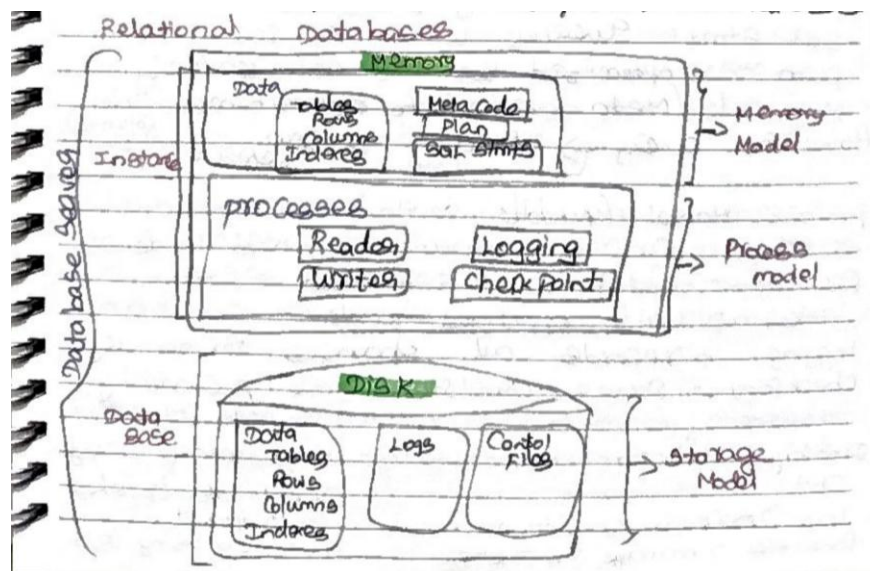
- Both mean the code is available and anyone can use it — the difference is the underlying perspective (technical-development vs. philosophical/freedom).

5.2 SQL

- **Non-procedural (declarative) language** — this is the key difference from Python/R.
- Older than Python & R — developed in 1974 at IBM.
- SQL behaves like an interpreter between you and the database.

5.3. Inside a Relational Database: The Three Internal Models

Layer	Role	Key Components
Memory Model	Where active work happens (RAM)	Data (Tables, Rows, Columns, Indexes); Metadata (Plan, SQL statements)
Process Model	Background operations	Readers, Writers, Logging, Checkpoint
Storage / Disk Model	Permanent storage	Data (Tables, Rows, Columns, Indexes), Logs, Control



From my hand-written notes: Memory, Process & Disk (Storage) models of a relational database.

5.4 How It Works

- **Memory layer:**
 - **Data section** tables are structured like spreadsheets — rows = records, columns = attributes; indexes speed up searching;
 - **SQL statement** — queries written by user;
 - **Plan** - the optimized way to execute a query;
 - **Metadata** - info about schema.
- **Process layer:**
 - **Readers** fetch data from disk;
 - **Writers** write changes to disk;
 - **Logging** records all changes (for recovery);
 - **Checkpoint** saves a consistent state periodically.
- **Disk layer:** permanent storage — Data, Logs (track all transactions) & Control files (manage DB structure).
- Flow: SQL query → execution plan → data retrieval.

Key Takeaway

A relational database is NOT just tables. It is a system that: stores data on disk → loads it into memory for processing → uses processes to read/write/manage it → executes queries using optimized plans.

6. Libraries, APIs, Datasets & ML Models

6.1 Libraries — Collections of Pre-Built Code

Purpose	Libraries
Data handling	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Machine Learning	Scikit-learn
Deep Learning	Keras, TensorFlow, PyTorch
Big Data	Apache Spark

6.2 APIs

- **API** = an interface between two systems — a bridge that lets two software systems communicate.
- Example: functions like `read_csv()` and `mean()` are part of the Pandas API — they interface between user-written code and the library's internal implementation.

6.3 Datasets

- **Dataset**: a collection of data.
- **Tabular data**: each row = an observation; each column = info about that observation.
- A dataset can be a structured collection of text, numbers, images, audio or video.

6.4 Common Data Formats

Format	Examples
Tabular (rows & columns)	CSV files
Hierarchical / tree-structured	XML, JSON
Network / graph	Data showing relationships, e.g. social networks

6.5 Open Data — Limitations

- **Open datasets might not meet enterprise requirements.**
- Even though open data plays a major role in AI, ML & DS, companies often cannot rely on it directly because of:
 - Data quality issues — not reliable enough for business decisions.
 - Lack of specificity — open data is general-purpose, not tailored.
 - Privacy & security concerns.

6.6 ML Models

- **ML** uses algorithms (models) to identify patterns in data.

- **Model training:** the process by which a model learns these patterns from data.
- Traditional data analysis: a person manually inspects the data, or a specialized program automates human analysis — both approaches are limited by the amount/complexity of data.
- ML models, by contrast, learn from data themselves to make predictions.

6.7 Model Asset eXchange (MAX)

- Developed by IBM as an open-source platform.
- Provides pre-trained deep learning models, ready to use.
- Hosted on GitHub under the CODAIT organization (e.g. CODAIT/MAX-<model-name>).

8. Jupyter Notebook & JupyterLab

8.1 Jupyter Notebook

- Originated as “IPython” (Python only).
- Later extended to support Julia & R too — the name “Jupyter” = Julia + Python + R.
- “**Browser-based application**” = software that runs inside a web browser — you don't install/open it like a normal desktop app.
- Allows combining text, code & output in a single file.
- Notebooks can be exported to HTML / PDF so they can be shared with anyone.

8.2 JupyterLab

- **JupyterLab** = the full workspace environment — an advanced version of Notebook that supports working with multiple notebooks at once.

8.3 The Kernel

- **Kernel**: the computational engine that actually runs your code inside a Jupyter notebook.
- Jupyter supports multiple languages via kernels — each language has its own kernel (Python kernel, R kernel, Julia kernel).

You cannot mix languages in the same kernel — Python code needs a Python kernel, R code needs an R kernel.

- “No kernel” / notebook shutdown = the kernel is no longer active.

8.4 Jupyter Architecture

Component	Role
Client / Front-end	What the user sees in the browser — displays the notebook, sends code to the kernel, shows output
Kernel / Back-end	Backend execution engine — runs the code, returns the result
Notebook Server	Middle layer — saves/loads notebooks, handles communication between client & kernel

8.5 Notebook File & Saving

- **File format**: saved as a .ipynb file — a text-based, JSON-structured file.
- Save process: click Save → notebook sent to server → server stores the file on disk as .ipynb.
- **File conversion**: Jupyter provides the “nbconvert” tool to convert notebooks into other formats (HTML, PDF, Markdown).

8.6 Cloud-Based Jupyter Environments

Cloud-based environments used to create and modify jupyter notebooks I;e Jupyter Lite and google colab.

Jupyter Notebook	JupyterLite
Browser is just the interface; code executes on a local machine or remote server	Browser is the entire system — runs fully inside the browser

9. Git & GitHub — Version Control Basics

9.1 Why Version Control?

- **VCS (Version Control System)**: widely used among developers & DS professionals to version-control source code/projects and collaborate with others.
- Keeps track of changes to your documents.
- Makes it easy to recover older versions if you make a mistake.
- Makes collaboration with others much easier.

9.2 What Is Git?

- **Git**: free & open-source software.
- **Distributed VCS (DVCS)**: users anywhere in the world can have a copy of your project on their own computer.
- When they make changes, they can sync their version to a remote server to share it with you.

Git = VCS + “distributed” — that's why it's the most common VCS available.

9.3 Basic Git Workflow

1. Clone(copy) the project to your computer.
2. Make changes locally.
3. Save those changes (commit).
4. Send them to a shared place (remote) — often **GitHub**, **GitLab**, **Bitbucket** or Beanstalk.

9.4 Why “Distributed” Is a Big Deal

Benefit	Why It Matters
Safety	If one copy is lost, others still exist
Speed	Most operations happen locally
Flexibility	People can work independently & merge later
Collaboration	Multiple people can work without blocking each other

9.5 Git vs GitHub

- **Git** (the tool) runs on your computer — you CAN use Git without GitHub, just via the command line. You only need a shared place when you want to sync/share.
- **GitHub** is a website that hosts Git repositories. With Git alone you track changes locally; to sync with others you need to share your repository — GitHub is just one convenient way to do that.
- Git supports multiple ways to share:
 - Using a hosting service — GitHub, GitLab, Bitbucket.
 - Direct sharing (no GitHub at all) — sending the project folder (USB/email), or using Git directly over a network (SSH).

10. Git Core Workflow & Working with Branches

10.1 Git Is Not Just for Code

- Git is heavily used to track: code, documents (PDF, Word), images, and research files (datasets, experiment logs).

10.2 Key Vocabulary

Term	Meaning
SSH protocol	A method for secure remote login from one computer to another
Repository (Repo)	Your project folder + Git's tracking system — the folder set up for version control
Fork	A copy of someone else's repo, under your own account
Pull Request (PR)	Asking someone to review & approve your changes before they become final
Working Directory	The actual files & folders on your computer that you are editing
Clone	Copying a repository from somewhere (e.g. GitHub) to your local machine

10.3 Fork vs Clone

If It's Your Repo	If It's Someone Else's Repo (NO permission)
1. Clone it to start working (no fork needed)	1. Fork it on GitHub 2. Clone your fork locally 3. Make changes 4. Push to your fork 5. Open a Pull Request

10.4 The 3 Main Areas in Git

- With Git, everything revolves around 3 stages:

Area	What Happens	Command to Check
Working Directory	Where you edit files, on your local machine	git status
Staging Area	Where you prepare changes before saving	git add <file>
Repository	Where changes are permanently saved (committed)	git commit -m "message"

10.5 Core Commands

- **git status:** shows the state of the working directory & staged changes.
- **git add <file>:** moves selected changes to the staging area — “I want to include this in the next snapshot.”

- **git commit -m "msg"**: saves a snapshot into history permanently — “save this version permanently.”
- **git log**: views history — shows who made changes, when, and the commit message.
- **git reset**: undoes / unstages changes.

10.6 Branching & Merging

- **Branch**: a parallel version of your project — lets you work separately without affecting the main version.
- **Main branch**: the stable, official version.
- **Child branch**: where you experiment safely. Files created in a child branch don't exist in the main branch until merged.

Without branches: you edit directly on main → risky, bugs break everything. With branches: you experiment safely & main code stays stable.

- **Merge**: combines your branch's work back into the main branch once you're satisfied (git merge).

10.7 Pull Requests on GitHub

- Instead of merging directly, on GitHub you:
 5. Push your branch.
 6. Create a Pull Request (PR).
 7. Others review your changes.
 8. Then merge — **“Please review my changes before adding them to main.”**